Metastable Success: High Accuracy with a Diverging Forward Map

Pre-Expert Normalization and Silent Forward-Map Divergence in Routed and Weight-Tied Residual Networks

Abstract. A model reaching high accuracy is not evidence that its forward map is bounded. We trained a routed residual network to 59.8% exact match on character-level arithmetic while its residual stream grew **292-fold** between the first and last level — a superlinear, diverging profile that terminated in bf16 overflow within a few hundred steps once the routing temperature was raised. The cause is one missing LayerNorm: routed experts read the raw residual stream while our dense comparator LayerNorms its FFN input. Because SwiGLU responds to its input roughly as $\|x\|^2$, each level's update scales with the state it is added to, giving RMS(x_{l+1}) ≈ RMS(x_l)·(1 + c·RMS(x_l)) — a recursion that diverges at finite depth. We establish causality five ways: divergence occurs at random initialization with no optimizer step; a 2×2 ablation over {shared, per-level} × {norm, no-norm} shows both cell types diverge without the norm and neither does with it; fp32 does not prevent it; halving the expert output init scale moves the divergence threshold monotonically as the recursion predicts; and NaN incidence drops from 10.34% of logged steps to **0/234** when the norm is added. The trigger is routing sharpness, which sets c_eff — all eight logged NaNs occur at temperature ≥ 1.50, exactly where the anneal crosses that value. We then show weight tying's real cost is *update coherence* (cos ≈ 0.73 shared vs −0.01 per-level), independent of the numerical bug. We propose a pre-flight stability test: measure the initialization-time RMS(x) profile and refuse configurations above ~10^2, because float range will happily hide a diverging architecture for thousands of steps.

Budget note. Every accuracy figure comes from runs of 3,000–9,000 steps named in the reproducibility appendix; no claim here depends on comparing unequal training budgets. All costs are per item (MACs/token × measured mean sequence length), never per token alone.

---

1. The sentence you should remember

> **A model reaching high accuracy is not evidence that its forward map is bounded.**

This is the whole paper. Everything else is how we were fooled by it, how we caught it, and a cheap test so you do not have to be.

The concrete case: our best routed model scored 59.8% exact match — respectable, converging, no crashes — while the residual stream it computed on was growing like this (RMS(x) at each level of an 8-level network, measured on the *trained* checkpoint):

```
level      0     1     2     3     4     5      6       7
RMS(x)   0.1   0.99  1.59  3.15  4.36  6.30   16.35   289.49
```

A healthy residual network adds a roughly constant amount per layer; RMS(x) should grow slowly, sub-linearly in depth. Ours grew **292× from level 1 to level 7**, and the last three levels account for almost all of it. The same architecture at depth 12 does not survive: it reaches `inf` then `nan`.

We did not notice for weeks because bf16 has roughly 38 decimal orders of headroom, and because SGD is remarkably good at finding a stable basin *just below* the cliff.

2. The defect, read off the source

Two FFN call sites in the same repository:

```
# DenseBlock.forward                          # Cell.mix_soft / Cell.mix_hard
a, b = self.ff1(self.ln2(x))                  e_soft += p[..., i:i+1] * ex(a)
return x + self.ff2(F.silu(a) * b)            #                          ^ raw residual
        stream
```

`DenseBlock` LayerNorms before its FFN. The routed `Cell` did **not** LayerNorm before its experts — they read `a`, the accumulating residual stream. Attention was always fine (`qkv(self.ln1(x))`), and so was the auxiliary state path (`h_mlp(cat(ln2(a), ln3(h)))`). Only the routed experts were un-normalised. A `ln4` LayerNorm existed in the cell but was applied *after* the sum, and only under a non-default flag.

This is not a property of MoE, of weight tying, or of adaptive depth. It is one missing line in one module, and the dense model it was compared against did not have it.

3. Why this specific omission explodes

The expert is a SwiGLU MLP, `w2(silu(w1 x) · (w1 x))`. Both factors are linear in x and they are multiplied, so the output scales roughly as $\|\mathbf{x}\|^2$. Feed that an un-normalised input and each level's update grows with the very quantity it is being added to:

```
u_l  ≈  c · RMS(x_l)²
RMS(x_{l+1})  ≈  RMS(x_l) · (1 + c · RMS(x_l))                    (1)
```

where `c` absorbs the expert output-projection init scale and the routing gate. This recursion is super-exponential in depth — unlike linear residual growth, the multiplicative factor `(1 + c·RMS)` itself increases every level. Divergence at finite depth is not a training pathology; it is arithmetic.

3.1 The scaling law, measured inside one cell

The cleanest possible control exists inside a single cell at a single training step: attention reads `LayerNorm(x)`, the experts read raw `a`. Same module, same weights, same forward pass, same step — only the normalisation differs.

level	RMS(x_l)	attention update	expert update
1	0.44	0.771	0.083
2	0.92	0.757	0.216
3	1.44	0.787	0.452
4	2.12	0.786	0.815
5	3.00	0.773	1.408
6	4.23	0.750	2.813
7	6.42	**0.716**	**11.354**

Attention's contribution varies by a factor of **1.10** across all levels; the expert's by **25×**. Fitting the two candidate laws over levels with RMS(x) > 1 (coefficient of variation of the implied constant — lower is the better fit):

sublayer	CV of u/x	CV of u/x²	better fit
attention (reads `ln1(x)`)	**52.4%**	93.8%	u ~ x
experts (read raw `a`)	74.6%	**22.7%**	**u ~ x²**

The normalised path is flat; the un-normalised one follows the quadratic predicted by (1). *Reproduce:* `python3 scripts/scaling_law.py`.

4. It diverges at initialization, with no training at all

If this were an optimisation instability it should require gradients. It does not. Build each model at random initialization, feed one batch, take **no optimizer step**, record RMS(x) at the final level:

depth	shared cell, no LN	shared +LN	per-level cells, no LN	per-level +LN
4	1.8	1.5	1.28	1.21
6	8.4	2.8	2.6	1.8
8	**6.9e4**	4.1	**82**	2.3
12	**inf → nan**	7.0	**1.2e13 → inf**	3.1
16	**nan by level 11**	10.0	**nan by level 11**	3.7

Dense control (which has the norm): depth 8 → **2.22**, depth 16 → **3.46**. Flat, as theory requires.

The signature is clearest in how much each level *adds*, $|x_{l+1} - x_l|$:

configuration	added per level, first → last
per-level +**LN** (healthy)	0.88, 0.47, 0.66, 0.43, 0.85, 0.92, **1.00**
shared cell +**LN** (healthy)	bounded, roughly linear in depth
per-level cells, no LN	0.61, 0.51, 0.44, 0.32, 0.44, 0.37, 1.11, **24, 6499, 3.4e8**
shared cell, no LN	0.56, 0.65, 0.71, 1.03, **11.7, 1835, 5.1e7**

Reproduce: `python3 scripts/init_blowup.py`.

5. Causality: the 2×2 ablation

`Config.ln_expert` (now **default on**) inserts the LayerNorm on the expert input and nothing else. First non-finite step, 1500-step runs, identical seeds:

	no pre-expert LN	with pre-expert LN
per-level cells	diverges, step **335**	**stable, 1500 steps**
shared cell (weight-tied)	diverges, step **98**	**stable, 1500 steps**

Both cell types diverge without the norm; both are stable with it. **Weight tying is not the cause.** It is an *accelerator* (§7): a tied cell emits coherently aligned updates that add constructively, moving the failure from step 335 to step 98. *Reproduce:* `python3 scripts/ablate_blowup_cause.py`.

6. What it is not: three controls

Not precision. Running the diverging configuration in fp32 fails at step **99**; in bf16, step **98** — same trajectory (RMS 0.03 → 1.25 → 2.9 → 1467 → 3.7e7). Float range did not cause this and only delayed discovering it. Every "bf16 overflow" note in our earlier logs, including one embedded in the trainer's own NaN guard, misidentified the mechanism.

Not optimisation. It diverges at random initialization with no gradient step taken (§4), and before any learning-rate schedule, warmup, or data ordering is engaged.

It tracks the predicted constant. In recursion (1), `c` is set by the expert output projection's init std (`out_std`). Sweeping it — depth 12, tied cell, no norm:

out_std (= c)	predicted takeoff once RMS(x) > 1/c	final RMS(x) at level 12	diverged
0.0625	16	**nan**	yes
0.03125	32	**nan**	yes
0.015625	64	4.6e11	no (barely)
0.0078125	128	111.8	no
0.00390625	256	8.8	no

Halving `c` monotonically pushes the divergence threshold deeper, exactly as (1) predicts: smaller `c` means more levels before RMS(x) passes 1/c and takes off. This is a quantitative match to the mechanism rather than a correlation. *Reproduce:* `python3 scripts/mechanism_confirm.py`.

7. The trigger is routing sharpness, not depth

In gated routing the routed term is `gate_scale · p_sel · FFN(a)`, and `p_sel ≈ 1/E` when the router's softmax is cold. So **c_eff ∝ gate_scale · p_sel**: a cold router attenuates the superlinear feedback, a sharp one releases it. Trainers routinely *anneal temperature upward*, which drives any marginal model across its threshold mid-run.

Final-level RMS(x) at random initialization (no optimizer step), bf16:

d	depth	ln_expert	T=0.5	T=1	T=2	T=3	T=4	T=8	T=16
256	8	0	2.36	3.08	9.28	14.8	21.0	56.4	306
256	8	**1**	2.22	2.23	2.24	2.25	2.26	2.26	2.27
256	12	0	3.45	441	**inf**	inf	inf	inf	inf
256	12	**1**	2.98	2.98	2.99	2.99	2.98	2.98	2.97
128	12	0	3.81	8.0e10	**inf**	inf	inf	inf	inf
128	12	**1**	3.03	3.04	3.07	3.08	3.09	3.09	3.09

With the norm, RMS(x) is essentially **independent of routing temperature** — it moves ~2% across a 32× change in T, because every expert sees unit-variance input regardless of which expert is selected or how sharply. Without it, the same quantity spans 3.8 → 8e10 → inf. The LayerNorm does not merely raise a threshold; it **decouples routing sharpness from residual magnitude**.

7.1 Every logged NaN in the corpus lands where the mechanism predicts

All eight non-finite events across 53 pre-fix fractal runs, with the annealed temperature at the moment of failure:

run	depth	gate_scale	step of first NaN	temperature there
MX_L	12	1.0	400	**1.50**
STRICT_d128	8	1.0	400	**1.50**
MX_full	12	1.0	250	**2.37**
D_st	8	1.0	500	3.62
D_stgate	8	1.0	500	3.62
SAN_e8_gs8	8	**8.0**	500	5.18
D_soft	8	1.0	1000	6.74
C1_easy	8	1.0	2500	8.00

Ordering predictions, all satisfied: no NaN below T = 1.50; the two depth-12 runs fail earliest (1.50, 2.37) because depth compounds the recursion; `SAN_e8_gs8`, whose `gate_scale=8` multiplies c_eff eightfold, fails well before its `gate_scale=1` siblings at comparable width and depth. The two T = 1.50 events are both at step 400 of a 3000-step anneal — the first logged sample after the schedule crosses 1.5 (temp(400) = 0.5 + 7.5·400/3000 = 1.500).

Reproduce: `python3 scripts/temp_coupling.py`.

8. Metastability: the model that worked anyway

RMS(x) per level, measured on **trained** checkpoints (`scripts/trained_residual_profile.py`):

run	pre-expert LN	tied	depth	exact	RMS(x) by level	final / level-1
D128_long	no	no	8	**59.8%**	0.1, 1.0, 1.6, 3.1, 4.4, 6.3, 16.4, **289**	**292×**
LNE_D128_long	yes	no	8	60.0%	0.1, 0.9, 1.6, 2.6, 4.7, 5.8, 7.1, **9.5**	10×
STRICT_d128	no	yes	8	0.0%	0.0, 0.7, 1.3, 2.7, 11.1, 515, 2.6e6, **5.9e13**	**9.0e13×**
LNE_STRICT	yes	yes	8	48.4%	0.1, 1.2, 2.9, 5.2, 7.9, 10.7, 13.6, **16.5**	14×
LNE_d12	yes	no	12	59.3%	…linear…	16.5×
MX_L	no	no	12	53.7%	0.1, 0.9, 1.5, 1.9, 2.1, 2.4,	9.9×

run	pre-expert LN	tied	depth	exact	RMS(x) by level	final / level-1
					2.6, 3.0, 3.4, 4.1, 4.9, **9.0**	

`D128_long` is the row to stare at: **59.8% accuracy on a forward map that has grown 292-fold by its last level.** It is not healthy; it is *metastable*. Training settled into a basin below the overflow point, and bf16's ~38 orders of headroom absorbed an excursion or two rather than ending the run.

This also unifies the two NaN behaviours we initially thought were separate bugs. **Transient** NaN — one step goes non-finite, the guard skips the update, training continues (MX_L NaN'd at step 400 and finished at 53.7%). **Terminal** NaN — weights die permanently (STRICT_d128). These are one mechanism sampled at different distances from the threshold: a model near the edge absorbs occasional excursions; one that crosses it cannot return.

9. The fix, and what it is worth

run	tied	params	outcome
STRICT_d128	yes	490K	**NaN at step 400, dead.** Best answer accuracy 0.8%
LNE_STRICT	yes	490K	stable for all 9,000 steps, **48.4% exact / 48.8% answer**
D128_long	no	3.75M	transient NaNs, 59.8% / 60.6%
LNE_D128_long	no	3.75M	stable, **60.0% / 61.1%**

One LayerNorm converts an untrainable configuration into a learning one — weight tying at 490K parameters (a 7.7× reduction from 3.75M) simply did not run before it. Across the whole corpus:

configuration	fractal runs	logged steps	NaN steps	rate
pre-fix (`ln_expert=False`)	53	561	58	**10.34%**
post-fix (`ln_expert=True`)	10	234	**0**	**0.00%**

Two honest caveats. First, logged steps are sampled (every 50–500 steps), so these are lower bounds on NaN incidence, not exact counts; the 0/234 post-fix figure is a non-observation and is weaker evidence than the 2×2 ablation in §5. Second — and this matters for how the fix should be valued — **it moves downstream accuracy by only +0.2 exact / +0.5 answer points** (60.0/61.1 vs 59.8/60.6, both at best-by-exact checkpoint selection). The bug was real, it explained every NaN, and it blocked an entire architecture from training at all. It was **not** what was limiting accuracy.

10. Weight tying's real cost is coherence, not divergence

Independent of the numerical bug, tying weights across depth changes what the network *computes*. Cosine similarity between consecutive levels' updates, matched initialization, depth 8:

tied	pre-expert LN	mean cos(expert update_l, update_{l+1})	mean cos(attention update)
no	no	**−0.005**	+0.007
no	yes	−0.012	+0.020
yes	no	**+0.463**	+0.612
yes	yes	**+0.734**	+0.769

Independent cells emit near-orthogonal updates that partially cancel — which is precisely why they diverge later than a tied cell, and why the norm matters so much less for them (−0.005 → −0.012). A tied cell applies nearly the *same* transformation at every depth, so its oversized updates add constructively (+0.734) instead of cancelling.

Two consequences. Numerically, coherence is why tying turned a step-335 failure into a step-98 one (§5). Representionally, it is an expressiveness ceiling that no amount of normalisation removes: after the fix, the tied model reaches 48.4% and the untied model 60.0% at matched budget — an 11.6-point gap for 7.7× fewer parameters. That gap is the honest price of self-similarity, and it exists with the numerical bug gone.

11. A pre-flight stability test that costs one forward pass

Do not ask “did it train?” Ask “what does RMS(x) do at initialization?”

```
python3 scripts/init_blowup.py      # per-level RMS(x), no optimizer step, ~1 min
python3 scripts/growth_map.py       # 42-config sweep of the same quantity
```

Over 42 configurations (widths 128/256 × expert widths 64/128/256 × depths 4/8/12 × tied/untied), at **random initialization with no training**:

- **12 of 42 already overflow.** All 12 are depth-12 without the pre-expert norm; every one of them is a configuration we ran and debugged as if it were an optimisation problem.
- Survivors span RMS(x) from **1.2 to 4.6e12** — twelve orders of magnitude of very different behaviour, all labelled “it trained.”

Concretely: print per-level RMS(x) at initialization and refuse the run above ~10^2. Healthy profiles in this codebase sit near 1–3 (dense reaches only 3.46 at depth 16). This test costs one forward pass on one batch and would have caught every failure in this paper before any GPU time was spent.

12. Scope and what remains open

Demonstrated for SwiGLU experts reading an un-normalised residual, in a routed/weight-tied transformer at 0.5–9M parameters, on character-level arithmetic. The mechanism (superlinear sublayer + un-normalised accumulating input) is generic; the specific constant `c` is not.

Not established: whether RMS(x) growth of this kind degrades *quality* before it degrades *validity* — our fixed model gained only 0.2 accuracy points, so within the range tested, metastability appears nearly free in accuracy terms while being very expensive in robustness. Whether a task with longer horizons or tighter precision (fp16, int8) converts this into an accuracy loss is untested and is the obvious next question. We also did not test whether other superlinear sublayers (GLU variants, gated convolutions,

squaring attention variants) share the same threshold behaviour; we expect they do, since the argument only uses $\|u(x)\| \sim \|x\|^2$.

13. Context

The routing hypothesis that motivated this investigation — whether adaptive-depth, weight-tying architectures can beat dense transformers at matched compute — is evaluated separately in **Paper 1** ("Parameter Sharing Is Not Compute Sharing"). The stability finding reported here is independent of that verdict: fixing every defect documented above changes the best accuracy by 0.2 points and does not change Paper 1's conclusion.

Appendix A. Reproducibility

Environment: Python 3.14.7, PyTorch 2.11.0+cu130, NVIDIA RTX 3080 10GB, AMD Ryzen 9 5900X. From the repository root: `export PYTHONPATH=src`.

A.1 Scripts (each writes JSON under `research/results/`)

command	produces	paper section
`python3 scripts/scaling_law.py`	within-cell u~x² fit, CV table	§3.1
`python3 scripts/init_blowup.py`	init-only RMS(x) profiles + dense control	§4
`python3 scripts/ablate_blowup_cause.py --share/--ln-expert/--fp32 --steps 1500 --tag T`	2×2 causal ablation → `abl_T.json`	§5
`python3 scripts/mechanism_confirm.py`	out_std (constant c) sweep + update coherence	§6, §10
`python3 scripts/temp_coupling.py`	temperature grid + all observed NaN onsets	§7
`python3 scripts/trained_residual_profile.py`	per-level RMS(x) on trained checkpoints	§8
`python3 scripts/growth_map.py`	42-configuration growth surface	§11
`python3 scripts/probe_shared_cause.py [--share] --amp`	original discovery instrumentation	§3, §8
`python3 scripts/paper_tables.py`	renders P2-T1…T8 exactly as quoted here	all

The 2×2 grid reproducing §5:

```
for spec in "A::" "B:--ln-expert:" "C:--share:" "D:--share --ln-expert:"; do
  tag=${spec%%:*}; rest=$(echo $spec | cut -d: -f2)
  python3 scripts/ablate_blowup_cause.py $rest --steps 1500 --tag $tag &
```

```
done
python3 scripts/ablate_blowup_cause.py --share --fp32 --steps 1500 --tag C_fp32
```

A.2 Checkpoints referenced

path	what it is
`runs/D128_long/`	pre-fix, per-level cells, depth 8 — the metastable 59.8% model (292× growth)
`runs/LNE_D128_long/`	same + `--ln-expert`, 60.0%, 10× growth
`runs/STRICT_d128/`	pre-fix weight-tied, dead at step 400
`runs/LNE_STRICT/`	weight-tied + fix, 48.4%, trainable for the first time
`runs/MX_L/`, `runs/MX_full/`, `runs/C1_easy/`, `runs/D_soft/`, `runs/D_st/`, `runs/D_stgate/`, `runs/SAN_e8_gs8/`	pre-fix runs contributing the eight logged NaNs of §7.1

Each directory contains `config.json` (full `Config` + CLI args, including `ln_expert`), `best.pt`, and `log.jsonl`. Pre-fix checkpoints omit `ln_expert`; loaders default it to `False` via `config_from()` so they still load into current code.

A.3 The fix in the code

`src/hmoe/model.py`: `Config.ln_expert` (default **True**), `Cell.ln_e`, and `Cell.expert_in(a)`, applied at all three expert call sites (`mix_soft`, `mix_hard`, `mix_all_forced`). Pass `--no-ln-expert` to reproduce any pre-fix run bit-for-bit. The ablation scripts never modify the model file — they toggle this flag.

A.4 Conventions used in every table here

Cost is MACs **per item** = MACs/token × mean(prompt+target) tokens measured from that dataset split's own metadata; sequence length is never assumed. Accuracy is best-by-exact-match, applied identically to both model families. Training budget (3,000 or 9,000 steps) is stated per run in §A.2 rather than implied. NaN rates count *logged* steps, which are sampled — they are lower bounds.